

Estado compartilhado

Quando o dado não cabe dentro de uma tela só

Prof. Cristiano Rodrigues

A tela de resumo precisa da lista de hábitos. Por onde ela chega?

As duas estão lado a lado na barra inferior que vocês montaram.



Para acompanhar hoje

Você precisa saber

- Guardar e atualizar estado dentro de uma tela com `setState`
- Liberar o que tem controller no `dispose`
- Ir para outra tela e voltar, levando o dado no construtor

Você precisa ter feito

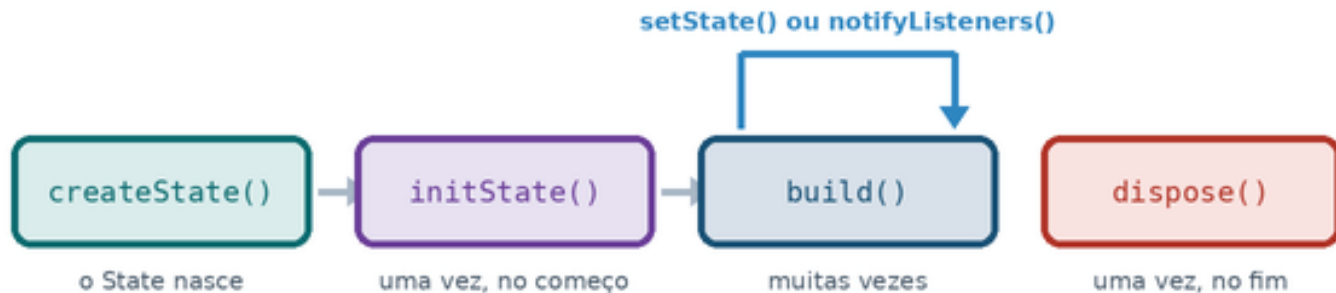
- O projeto separado em duas telas, com a lista e o formulário
- O hábito novo chegando na lista pelo retorno da outra tela
- A barra inferior com as seções do produto, ainda que com telas vazias

PARTE 01

O ciclo de vida de uma tela

Toda tela com estado nasce, é desenhada muitas vezes e morre.

Nasce uma vez, desenha muitas, morre uma vez



Só o build roda muitas vezes. Nada que custa caro pode ficar dentro dele.

Cada método tem um momento, e usar o momento errado custa caro.

Os três momentos, num exemplo só

```
class _TelaDetalheState extends State<TelaDetalhe> {  
  late final TextEditingController _meta;  
  @override  
  void initState() {  
    super.initState();           // sempre primeiro  
    _meta = TextEditingController(text: widget.habito.meta);  
  }  
  
  @override  
  void dispose() {  
    _meta.dispose();           // sempre antes  
    super.dispose();  
  }  
}
```

lib/tela_detalhe.dart · o campo já abre com a meta atual, e nada vaza ao sair

O build pode rodar dezenas de vezes por segundo.

Ele é chamado a cada reconstrução, e você não controla quando. Carregar dados, abrir arquivo ou chamar a internet dentro dele trava o aplicativo. Esse tipo de coisa vai no initState, que roda uma vez.



PARTE 02

Quando o dado não é de uma tela só

O construtor resolve duas telas. A partir daí ele começa a cobrar.

Duas maneiras de a lista chegar onde precisa

Passando por construtor



A tela do meio carrega o que não usa.

Com a lista acima das telas



Cada tela pega direto o que precisa.

À esquerda, cada tela repassa. À direita, cada tela pega.

A lista virou uma classe, e ela avisa quando muda

```
import 'package:flutter/foundation.dart';

class Habitostore extends ChangeNotifier {
  final List<Habitostore> _habitostore = [];

  List<Habitostore> get habitostore => List.unmodifiable(_habitostore);

  void adicionar(Habitostore h) {
    _habitostore.add(h);
    notifyListeners();          // quem estiver ouvindo, redesenhe
  }
}
```

lib/habitostore.dart · a lista sai das telas e passa a viver numa classe só

Instalar e pendurar a loja acima das telas

```
// no terminal, uma vez:
flutter pub add provider

// em lib/main.dart, e em toda tela que usar a loja:
import 'package:provider/provider.dart';

void main() => runApp(
  ChangeNotifierProvider(
    create: (_) => Habitostore(),
    child: const MeuApp(),
  ),
);
```

lib/main.dart · a loja passa a existir acima de todas as telas do aplicativo

PARTE 03

Ler e escrever na loja

Duas formas de pegar a loja, e a diferença entre elas decide o que redesenha.

watch e read: a diferença é uma só

context.watch

- Pega a loja e se inscreve para ser avisado.
- O widget reconstrói quando a loja notifica.
- Use no build, para mostrar o dado.

context.read

- Pega a loja uma vez, sem se inscrever.
- O widget não reconstrói por causa dela.
- Use dentro de onPressed, para mandar mudar.

A tela lê a lista, e o botão manda adicionar

```
import 'package:provider/provider.dart'; // uma vez por arquivo

// no build, para mostrar:
final habitos = context.watch<HabitosStore>().habitos;
return ListView.builder(
  itemCount: habitos.length,
  itemBuilder: (_, i) => ListTile(title: Text(habitos[i].nome)),
);

// dentro do onPressed, para mudar:
onPressed: () {
  context.read<HabitosStore>().adicionar(novo);
},
```

lib/tela_habitos.dart · ao adicionar, a lista redesenha sozinha, sem setState

Could not find the correct Provider above this Widget

É o erro mais comum, e ele diz a verdade: o provider não está acima de quem procura. Quase sempre foi pendurado dentro da tela em vez de acima do aplicativo. Suba ele no main e o erro some.

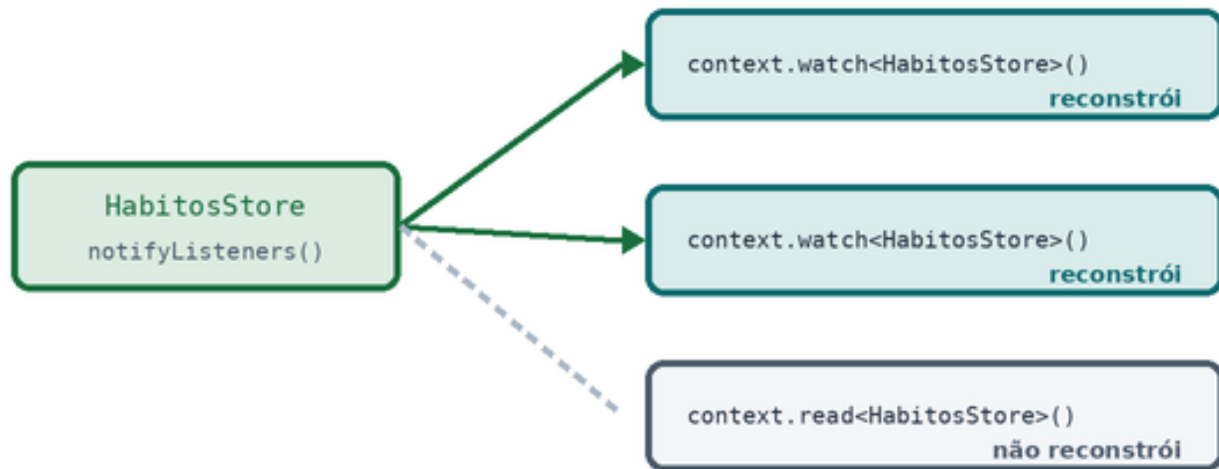


PARTE 04

Quem redesenha, e quando

Notificar todo mundo é simples. Notificar só quem precisa é o passo seguinte.

Quem escuta reconstrói, quem só pega não



A mesma loja, e dois comportamentos diferentes conforme quem pediu.

Consumer: reconstruir só um pedaço

```
Column(  
  children: [  
    const CabecalhoPesado(),          // não depende da loja  
  
    Consumer<HabitosStore>(  
      builder: (context, loja, filho) =>  
        Text('${loja.habitos.length} hábitos'),  
    ),  
  ],  
)
```

lib/tela_resumo.dart · o número atualiza sozinho, e o cabeçalho não redesenha

Onde o Provider se encaixa entre as opções

setState

O estado é de uma tela só. Continua sendo a resposta certa.

Provider

O estado é de várias telas. É o que esta disciplina usa.

Riverpod

O mesmo problema, sem depender do context. Muito comum hoje.

Bloc

Eventos e estados explícitos. Equipe grande, regra complexa.

Trocar de ferramenta é decisão de projeto, não de gosto. Saber o nome já evita cair de paraquedas.

PARTE 05

A prática

Tirar a lista de dentro da tela e pendurar acima de todas.

A refatoração, passo a passo

1. Criar

Uma classe `ChangeNotifier` com a lista privada e o método `adicionar`.

2. Pendurar

`ChangeNotifierProvider` envolvendo o aplicativo, dentro do `main`.

3. Ler

Na tela da lista, `context.watch` no lugar do campo do `State`.

4. Escrever

No botão `Salvar`, `context.read` e `adicionar`, sem `dado` no `pop`.

5. Limpar

O método que abria o formulário não espera mais resultado.

O que muda no código que você já tem

A lista

Sai do State e passa a viver na loja, acima das telas.

O setState

Sai da tela e vira notifyListeners dentro da loja.

O pop

Para de carregar dado. Volta a apenas fechar a tela.

O reinício

O hábito continua sumindo: nada foi gravado ainda.

Três linhas mudam de lugar. Nenhuma funcionalidade nova entra hoje.

O que a Sprint pede, e o que você já tem

Já tem

Telas que conversam

A lista acima das telas, e qualquer uma lendo dela.

Já tem

Tela redesenhando sozinha

notifyListeners avisando quem estiver mostrando o dado.

Falta

O dado sobreviver ao fechar

Enquanto nada for gravado, tudo recomeça do zero.

Com o estado num lugar só, gravar vira mexer em um arquivo, e não em todas as telas.

Nesta aula, você aprendeu a:

- Situar initState, build e dispose no ciclo de vida de uma tela
- Justificar quando o estado deixou de caber dentro de uma tela
- Escrever uma classe ChangeNotifier que avisa quem depende dela
- Pendurar a loja acima das telas com ChangeNotifierProvider
- Decidir entre watch e read pelo que o widget faz com o dado

Referências

Flutter. Simple app state management. docs.flutter.dev/data-and-backend/state-mgmt

Flutter. List of state management approaches. docs.flutter.dev/data-and-backend/state-mgmt/options

pub.dev. provider: documentação e exemplos do pacote.

Catálogo de componentes de interface. Deck de referência da disciplina, no Canvas.